Assignment 1

Name: Riddhi Agarwal
Roll Number: 2024CSB046

Problem 1

Question

Write a C program that generates n random integers, where the value of n is provided by the user, and stores these integers in a file. Then, read all the elements from the file and display them on the screen. Finally, determine and print the maximum and minimum values from the list.

Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main()
{
    srand(time(NULL));
    int n = 0;
    printf("Enter the value of n:");
    scanf("%d", &n);
    FILE *fp;
    fp = fopen("integers.txt", "w");
    if (fp == NULL)
    {
        printf("Error opening file.\n");
        return 0;
    }
    for (int i = 0; i < n; i++)
    {
        int rand_num = rand();
        fprintf(fp, "%d ", rand_num);
    }
    fclose(fp);
    fp = fopen("integers.txt", "r");
    if (fp == NULL)
    {
        printf("Error opening file.\n");
        return 0;
    }
    int min = RAND_MAX;
    int max = 0;
```

```c
    int num = 0;
    printf("Reading elements from the file:\n");
    while (fscanf(fp, "%d", &num) != EOF)
    {
        printf("%d ", num);
        if (num > max)
        {
            max = num;
        }
        if (num < min)
        {
            min = num;
        }
    }
    fclose(fp);
    printf("\nMaximum value = %d.\n", max);
    printf("Minimum value = %d.\n", min);

    return 0;
}
```

Output

```
PS C:\Users\USER\code\College Coding\Semester 4\Algorithm Lab\Assignment 1> gcc Ques1.c
PS C:\Users\USER\code\College Coding\Semester 4\Algorithm Lab\Assignment 1> ./a.exe
Enter the value of n:5
Reading elements from the file:
23198 19291 21463 29304 21963
Maximum value = 29304.
Minimum value = 19291.
PS C:\Users\USER\code\College Coding\Semester 4\Algorithm Lab\Assignment 1> cat .\integers.txt
23198 19291 21463 29304 21963
PS C:\Users\USER\code\College Coding\Semester 4\Algorithm Lab\Assignment 1>
```

Figure 1: Output for Problem 1.

Problem 2

Question

Write a C program to generate an array of 50,000 random elements. Implement both Linear Search and Binary Search algorithms to search for a given element in the array. If the element is found, display its position. Finally, compare the two search methods by reporting the number of comparisons performed and the time taken to locate the element.

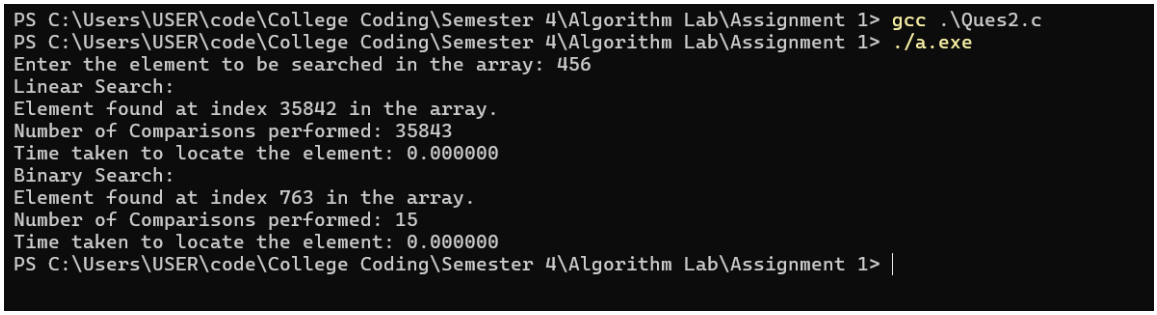
Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

void bubbleSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (arr[j] > arr[j + 1])
                swap(&arr[j], &arr[j + 1]);
}

int main()
{
    srand(time(NULL));
    int size = 50000;
    int arr[size];
    for (int i = 0; i < size; i++)
    {
        arr[i] = rand();
    }
    int num;
    printf("Enter the element to be searched in the array: ");
    scanf("%d", &num);
    // Linear Search
    clock_t start, end;
    int comp1 = 0;
    int idx = -1;
    double t1;
    start = clock();
    for (int i = 0; i < size; i++)
    {
```

```
        comp1++;
        if (arr[i] == num)
        {
            idx = i;
            break;
        }
    }
    end = clock();
    t1 = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("Linear Search:\n");
    if (idx != -1)
    {
        printf("Element found at index %d in the array.\n", idx);
    }
    else
    {
        printf("Element not found in the array.\n");
    }
    printf("Number of Comparisons performed: %d\n", comp1);
    printf("Time taken to locate the element: %f\n", t1);
    // Binary Search
    idx = -1;
    double t2;
    int comp2 = 0;
    bubbleSort(arr, size);
    int left = 0;
    int right = size - 1;
    start = clock();
    while (left <= right)
    {
        int mid = (left + right) / 2;
        if (num < arr[mid])
        {
            right = mid - 1;
        }
        else if (num > arr[mid])
        {
            left = mid + 1;
        }
        else
        {
            idx = mid;
            break;
        }
        comp2++;
    }
    end = clock();
    t2 = (double)(end - start) / CLOCKS_PER_SEC;
    printf("Binary Search:\n");
    if (idx != -1)
```

```c
    {
        printf("Element found at index %d in the array.\n", idx);
    }
    else
    {
        printf("Element not found in the array.\n");
    }
    printf("Number of Comparisons performed: %d\n", comp2);
    printf("Time taken to locate the element: %f\n", t2);
    return 0;
}
```

Output



```
PS C:\Users\USER\code\College Coding\Semester 4\Algorithm Lab\Assignment 1> gcc .\Ques2.c
PS C:\Users\USER\code\College Coding\Semester 4\Algorithm Lab\Assignment 1> ./a.exe
Enter the element to be searched in the array: 456
Linear Search:
Element found at index 35842 in the array.
Number of Comparisons performed: 35843
Time taken to locate the element: 0.000000
Binary Search:
Element found at index 763 in the array.
Number of Comparisons performed: 15
Time taken to locate the element: 0.000000
PS C:\Users\USER\code\College Coding\Semester 4\Algorithm Lab\Assignment 1>
```

Figure 2: Output comparison between Linear and Binary Search.

Analysis

Based on the output obtained for an array size of 50,000:

- **Linear Search:** The element was found at index 35,842. The algorithm required **35,843 comparisons**, which corresponds to $O(N)$ time complexity.
- **Binary Search:** The element was found at index 763 (in the sorted array). The algorithm required only **15 comparisons**, which corresponds to $O(\log N)$ time complexity.

It is evident that Binary Search is significantly more efficient than Linear Search in terms of the number of comparisons required.

Problem 3

Question

Write a C program that systematically creates ten separate arrays with sizes of 5000, 10000, 15000, 20000, 25000, 30000, 35000, 40000, 45000, and 50000 elements, respectively. Populate each array with randomly generated integer values. For every array, implement and apply four different sorting algorithms i.e., Bubble Sort, Insertion Sort, Quick Sort, and Merge Sort to arrange the elements in ascending order. Measure and record the execution time taken by each sorting algorithm for each array size, and finally perform a detailed comparison of their time performance to analyze and evaluate the efficiency of these sorting techniques as the input size increases.

Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

void bubbleSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (arr[j] > arr[j + 1])
                swap(&arr[j], &arr[j + 1]);
}

void insertionSort(int arr[], int n)
{
    for (int i = 1; i < n; i++)
    {
        for (int j = i; j > 0 && arr[j - 1] > arr[j]; j--)
        {
            swap(&arr[j], &arr[j - 1]);
        }
    }
}

void merge(int arr[], int l, int m, int r)
{ // l is left index, m is middle index and r is right index;
    int n1 = m - l + 1;
    int n2 = r - m;

```

```
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++)
        L[i] = arr[l + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];

    // merging left and right halves
    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2)
    {
        if (L[i] <= R[j])
            arr[k++] = L[i++];
        else
        {
            arr[k++] = R[j++];
        }
    }

    // copying remaining elements
    while (i < n1)
        arr[k++] = L[i++];
    while (j < n2)
        arr[k++] = R[j++];
}

void mergeSort(int arr[], int l, int r)
{
    if (l < r)
    {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

int medianOfThree(int arr[], int low, int high)
{
    int mid = (low + high) / 2;
    if (arr[low] > arr[mid])
        swap(&arr[low], &arr[mid]);
    if (arr[low] > arr[high])
        swap(&arr[low], &arr[high]);
    if (arr[mid] > arr[high])
        swap(&arr[mid], &arr[high]);
    swap(&arr[mid], &arr[high - 1]);
    return arr[high - 1];
}

int partition(int arr[], int low, int high)
{
```

```
    int pivot = medianOfThree(arr, low, high);
    int i = low, j = high - 1;
    while (1)
    {
        while (arr[++i] < pivot)
            ;
        while (arr[--j] > pivot)
            ;
        if (i >= j)
            break;
        swap(&arr[i], &arr[j]);
    }
    swap(&arr[i], &arr[high - 1]);
    return i;
}

void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

void copyArray(int src[], int dest[], int n)
{
    for (int i = 0; i < n; i++)
    {
        dest[i] = src[i];
    }
}

int main()
{
    srand(time(NULL));
    int sizes[] = {5000, 10000, 15000, 20000, 25000, 30000, 35000, 40000, 45000, 50000};
    int num_sizes = 10;

    printf("PERFORMANCE ANALYSIS OF SORTING ALGORITHMS\n");
    printf("==========================================================================\n");
    printf("| %-10s | %-12s | %-12s | %-12s | %-12s |\n", "Array Size", "Bubble (s)", "Insertion (s)", "Quick (s)", "Merge (s)");
    printf("==========================================================================\n");

```

```c
    for (int i = 0; i < num_sizes; i++)
    {
        int n = sizes[i];

        int *masterArr = (int *)malloc(n * sizeof(int));
        int *workArr = (int *)malloc(n * sizeof(int));

        if (masterArr == NULL || workArr == NULL)
        {
            printf("Memory allocation failed!\n");
            return 1;
        }

        for (int i = 0; i < n; i++)
        {
            masterArr[i] = rand();
        }

        double t_bubble, t_insert, t_quick, t_merge;
        clock_t start, end;

        // 1. Measure Bubble Sort
        copyArray(masterArr, workArr, n);
        start = clock();
        bubbleSort(workArr, n);
        end = clock();
        t_bubble = ((double)(end - start)) / CLOCKS_PER_SEC;

        // 2. Measure Insertion Sort
        copyArray(masterArr, workArr, n);
        start = clock();
        insertionSort(workArr, n);
        end = clock();
        t_insert = ((double)(end - start)) / CLOCKS_PER_SEC;

        // 3. Measure Quick Sort
        copyArray(masterArr, workArr, n);
        start = clock();
        quickSort(workArr, 0, n - 1);
        end = clock();
        t_quick = ((double)(end - start)) / CLOCKS_PER_SEC;

        // 4. Measure Merge Sort
        copyArray(masterArr, workArr, n);
        start = clock();
        mergeSort(workArr, 0, n - 1);
        end = clock();
        t_merge = ((double)(end - start)) / CLOCKS_PER_SEC;

        printf("| %-10d | %-12.6f | %-12.6f | %-12.6f | %-12.6f |\n",
               n, t_bubble, t_insert, t_quick, t_merge);
```

```c

        free(masterArr);
        free(workArr);
    }
    printf("
    ==========================================================================
    n");

    return 0;
}
```

Output

```
PS C:\Users\USER\code\College Coding\Semester 4\Algorithm Lab\Assignment 1> gcc .\Ques3.c
PS C:\Users\USER\code\College Coding\Semester 4\Algorithm Lab\Assignment 1> ./a.exe
PERFORMANCE ANALYSIS OF SORTING ALGORITHMS
==============================================================================
| Array Size | Bubble (s)   | Insertion (s) | Quick (s)    | Merge (s)    |
==============================================================================
| 5000       | 0.065000     | 0.050000      | 0.000000     | 0.000000     |
| 10000      | 0.329000     | 0.189000      | 0.000000     | 0.000000     |
| 15000      | 0.776000     | 0.420000      | 0.000000     | 0.000000     |
| 20000      | 1.471000     | 0.752000      | 0.000000     | 0.010000     |
| 25000      | 2.411000     | 1.204000      | 0.000000     | 0.009000     |
| 30000      | 2.725000     | 1.213000      | 0.004000     | 0.004000     |
| 35000      | 3.512000     | 1.660000      | 0.004000     | 0.004000     |
| 40000      | 4.731000     | 2.188000      | 0.000000     | 0.011000     |
| 45000      | 6.008000     | 2.702000      | 0.000000     | 0.011000     |
| 50000      | 7.479000     | 3.332000      | 0.010000     | 0.000000     |
==============================================================================
PS C:\Users\USER\code\College Coding\Semester 4\Algorithm Lab\Assignment 1>
```

Figure 3: Performance table for Bubble, Insertion, Quick, and Merge Sort.

Analysis

We tested four sorting algorithms with array sizes ranging from 5,000 to 50,000 elements. The results show a clear split in performance:

1. **Slow Algorithms (Bubble & Insertion Sort):**
    - As the array size increased, the time taken by Bubble and Insertion sort increased drastically.
    - Bubble Sort was the slowest, taking over 7 seconds for 50,000 elements. Insertion Sort was faster but still took over 3 seconds.
    - This shows that these methods are inefficient for large datasets.
2. **Fast Algorithms (Quick & Merge Sort):**
    - Quick Sort and Merge Sort were incredibly fast, taking nearly 0 seconds for all test cases.
    - Even with 50,000 elements, they finished almost instantly (0.01 seconds or less).
    - This proves that Quick Sort and Merge Sort are much better suited for handling large amounts of data.